МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Вычислительная математика»

Тема: Решение нелинейных уравнений. Метод хорд.

Студент гр. 4342 _____ Алюкин Г.А.

Преподаватель _____ Пуеров Г.Ю.

Санкт-Петербург

2025

Цель работы.

Пусть найден отрезок [a, b], на котором функция f. x. меняет знак. Для определенности положим f (a)>0, f (b)<0. В методе хорд процесс итераций состоит в том, что в качестве приближений к корню уравнения f. x. * 0 принимаются значения c0, c1, . . . точек пересечения хорды с осью абсцисс, как это показано на рис.1.

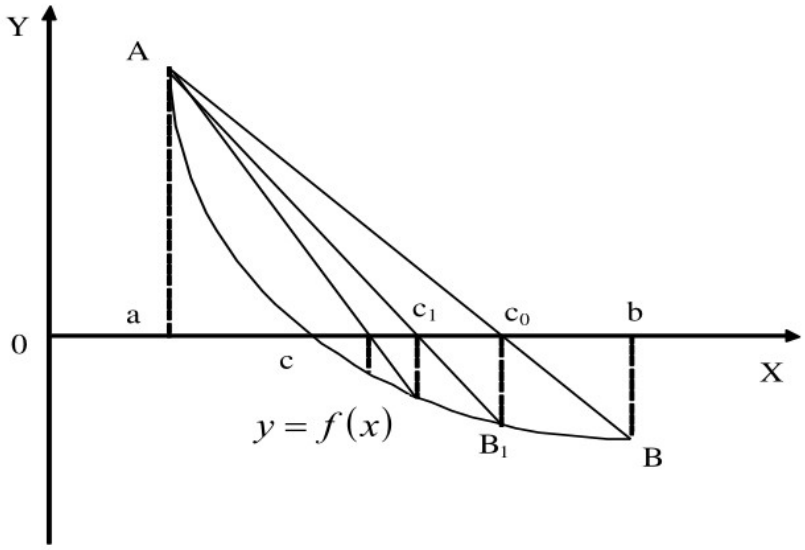

Рис. 1

Сначала находится уравнение хорды AB:

$$\frac{y-f(a)}{f(b)-f(a)}=\frac{x-a}{b-a}.$$

Для точки пересечения ее с осью абсцисс (x=c0, y=0) получается уравнение

$$c_0=a-\frac{b-a}{f(b)-f(a)}f(a).$$

Далее сравниваются знаки величин f (a) и f (c0) и для рассматриваемого случая оказывается, что корень находится в интервале (a, c0), так как f (a) f (c0)<0. Отрезок [c0,b] отбрасывается. Следующая итерации состоит в

определении нового приближения c1 как точки пересечения хорды AB1 с осью абсцисс и т.д. Итерационный процесс продолжается до тех пор, пока значение f (cn) не станет по модулю меньше заданного числа ε (см. подраздел 3.1).

Алгоритмы методов бисекции и хорд похожи, однако метод хорд в ряде случаев дает более быструю сходимость итерационного процесса, причем успех его применения, как и метода бисекции, гарантирован.

В лабораторной работе №4 предлагается, используя программы - функции HORDA и Round из файла methods.cpp (файл заголовков metods.h, директория LIBR1), найти корень уравнения f(x) = 0 с заданной точностью Eps методом хорд, исследовать скорость сходимости и обусловленности метода.

Для данной работы, как и для лабораторной работы №3 задаются индивидуальные варианты нелинейных уравнений (см. подраздел 3.6).

Порядок выполнения лабораторной работы №4:

1) Графически или аналитически отделить корень уравнения f(x) = 0 (т.е. найти отрезки [Left, Right], на которых функция f(x) удовлетворяет условиям применимости метода).

2) Составить подпрограмму - функцию вычисления функции f(x), предусмотрев округление значений функции с заданной точностью Delta с использованием программы Round.

3) Составить головную программу, вычисляющую корень уравнения f(x) = 0 и содержащую обращение к подпрограмме f(x), HORDA, Round и индикацию результатов.

4) Провести вычисления по программе. Теоретически и экспериментально исследовать скорость сходимости и обусловленность метода.

В подразделе 3.7 приводится текст программы - функции HORDA, предназначенной для решения уравнения f(x) = 0 методом хорд.

Выполнение работы.

1. Отделение корня уравнения (т.е. нахождение отрезков [Left, Right], на которых функция удовлетворяет условиям применимости метода).

Функция:

$$f(x)=x^2 – 5\sin(x)$$

Для отделения корней воспользуемся свойством непрерывной функции: если на концах отрезка функция имеет разные знаки, то внутри отрезка существует хотя бы один корень.

Подставляем разные значения x и смотрим на знак результата:

x	Результат f(x)	Знак
-1	≈ 5.2	+
0	0	Корень
1	≈ -3.2	-
2	≈ -0.55	-
3	≈ 8.3	+

Вычисляем значения функции:

$f(2)=2^2 – 5\sin(2) \approx -0.55$ - отрицательное

$f(3)=3^2 – 5\sin(3) \approx 8.30$ - положительное

- Непрерывность: Функция непрерывна как разность двух непрерывных функций:

- Производные:

$f'(x) = 2x - 5\cos(x)$. На [2,3] производная положительна ($2x >= 4$), значит функция монотонно возрастает.

$f''(x) = 2x + 5\cos(x)$. На [2, 3] синус положителен, значит $f''(x) > 0$. Функция не меняет направление выпуклости.

Между x = 2 (функция ниже нуля) и x = 3 (выше нуля) происходит изменение знака. Корень лежит на отрезке [2, 3], Функция должна быть непрерывна и менять знак. Это выполняется, следовательно берём данный отрезок в качестве начального для метода бисекции.

[Left; Right] = [2; 3].

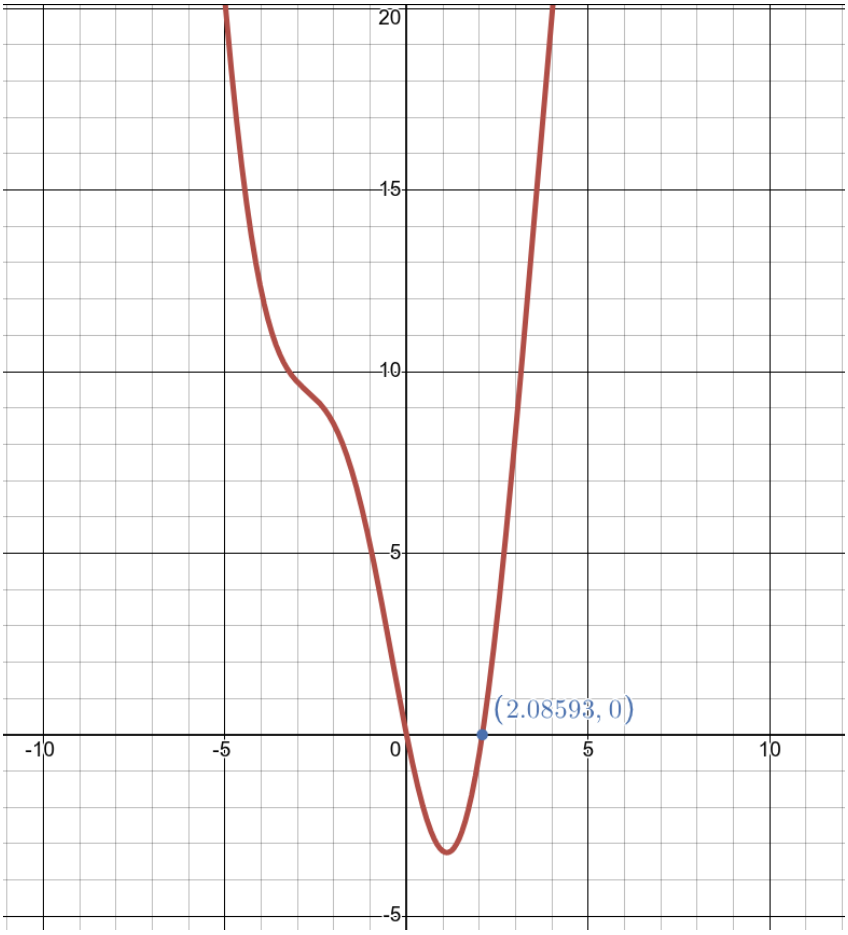

Рисунок 1 — График функции

2.Подпрограмма вычисления функции f(x).

Для работы метода бисекции необходима подпрограмма, вычисляющая f(x) для произвольного x. Эта функция вызывается на каждой итерации для определения значений в граничных точках и середине текущего интервала.

Создана подпрограмма находящаяся в файле lab4.cpp в Приложении А.

```
double F(double x){
    extern double delta;
    double s;
    s = x * x - 5.0 * sin(x); // Значение функции
    s = Round(s, delta); // Округление с точностью delta
```

```
    return s;
}
```

Функция F(double x) реализует вычисление значения исследуемого уравнения с имитацией погрешности вычислений. Результат вычисления передается в функцию Round, которая округляет его до шага, определяемого параметром delta.

3. Составление головной программы, содержащей обращение к подпрограмме f(x), HORDA, Round и индикацию результатов.

Создана головная функция `int main()` находящаяся в файле lab4.cpp в Приложении А.

4. Провести вычисления по программе. Теоретически и экспериментально исследовать скорость сходимости и обусловленность метода.

На основе выполнения алгоритма метода хорд получаем следующие результаты:

Точность (ε)	Корень (x)	Число итераций (N)
10^{-1} (0.1)	2.079011	1
10^{-2} (0.01)	2.085495	3
10^{-3}	2.085925	5
10^{-4}	2.085930	7
10^{-5}	2.085935	9
10^{-6}	2.085935	11

Построение графика зависимости N от Eps.

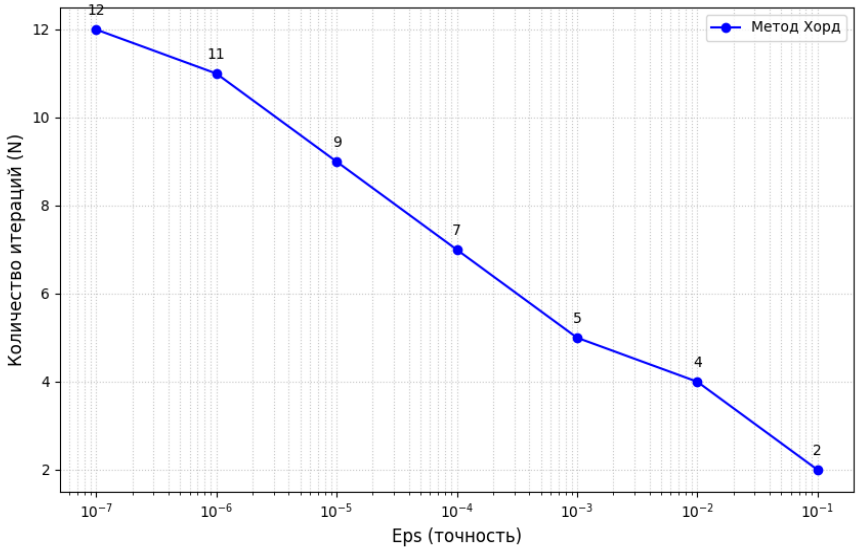

Рисунок 2 — График зависимости N от точности вычислений

5. Исследование чувствительности метода к ошибкам в исходных данных (моделирование Round)

Чтобы исследовать чувствительность метода к ошибкам в исходных данных, было проведено исследование с разными Delta и Eps. Результаты вычислений приведены ниже.

Delta	Eps	Значение HORDA	N
10^{-1}	10^{-1}	2.079011	1
	10^{-2}	2.079011	1
	10^{-3}	2.079011	1
	10^{-4}	2.079011	1
	10^{-5}	2.079011	1
	10^{-6}	2.079011	1
10^{-3}	10^{-1}	2.079413	2
	10^{-2}	2.085484	4
	10^{-3}	2.085815	5
	10^{-4}	2.085925	5
	10^{-5}	2.085925	5
	10^{-6}	2.085925	5
10^{-5}	10^{-1}	2.079413	2

	10^{-2}	2.085484	4
	10^{-3}	2.085815	5
	10^{-4}	2.085925	5
	10^{-5}	2.085925	5
	10^{-6}	2.085925	5
10^{-7}	10^{-1}	2.079372	2
	10^{-2}	2.085458	4
	10^{-3}	2.085807	5
	10^{-4}	2.085925	7
	10^{-5}	2.085934	9
	10^{-6}	2.085935	11

Исследование показало, что достижимая точность поиска корня напрямую ограничена погрешностью вычисления функции (Delta). Если требовать точность Eps выше, чем значение Delta, уточнение корня прекращается, а число итераций перестает расти.

Таким образом применение функции Round позволяет исследовать зависимость точности решения от ошибок округления. Сравнительный анализ результатов при изменении шага Delta дает возможность определить чувствительность алгоритма к точности представления чисел и выявить границы метода.

Вывод.

В ходе выполнения лабораторной работы был изучен и программно реализован метод хорд для нахождения корня нелинейного уравнения f(x)=x^2 – 5.0 * sin(x) на отрезке [2, 3]. Подтверждена линейная скорость сходимости метода: график зависимости числа итераций N от точности Eps является прямой линией. Также в ходе исследования было установлено, что на точность поиска корня влияет погрешность вычисления самой функции (Delta). Таким образом, метод бисекции является крайне надежным и устойчивым алгоритмом, однако его точность всегда ограничена качеством (точностью) задания исходных данных.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл lb4.cpp:

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include "methods.h"

double delta;

double F(double x){
    extern double delta;
    double s;

    s = x * x - 5.0 * sin(x); // Значение функции

    s = Round(s, delta); // Округление с точностью delta

    return s;
}

int main(){
    double a, b, eps, x;
    int k;
    float a1, b1, eps1, delta1;

    printf("Лабораторная работа №4. Вариант 1\n");
    printf("Функция: f(x) = x^2 - 5*sin(x)\n\n");

    printf("Введите границы интервала a и b: "); // 2 3
    scanf("%f %f", &a1, &b1);
    a = a1;
```

```
    b = b1;

    printf("Введите точность вычисления корня: "); // 0.0001
    scanf("%f", &eps1);
    eps = eps1; //Точность вычисления корня

    printf("Введите точность задания функции: "); // 0.000001
    scanf("%f", &delta1);
    delta = delta1; // Точность задания функции

    x = HORDA(a, b, eps, k);

    printf("\nРезультаты:\n");
    printf("x = %f\n", x);
    printf("Число итераций N = %d\n", k);
    return 0;
}
```

Файл methods.cpp:

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

extern double F(double);

double HORDA(double Left, double Right, double Eps, int &N)
{
    double FLeft = F(Left);
    double FRight = F(Right);
    double X, Y;
    if (FLeft * FRight > 0.0)
    {
        puts("Неверное задание интервала\n");
        exit(1);
```

```
    }
    if (Eps <= 0.0)
    {
        puts("Неверное задание точности\n");
        exit(1);
    }
    N = 0;
    if (FLeft == 0.0)
        return Left;
    if (FRight == 0.0)
        return Right;
    do
    {
        X = Left - (Right - Left) * FLeft / (FRight - FLeft);
        Y = F(X);
        if (Y == 0.0)
            return (X);
        if (Y * FLeft < 0.0)
        {
            Right = X;
            FRight = Y;
        }
        else
        {
            Left = X;
            FLeft = Y;
        }
        N++;
    } while (fabs(Y) >= Eps);
    return (X);
}
/*
ФУНКЦИЯ ОКРУГЛЕНИЯ
X - число
Delta - шаг округления (например, 0.01)
*/
```

```
double Round(double X, double Delta)
{
    if (Delta <= 1E-9) {
        puts("Неверное задание точности округления\n");
        exit(1);
    }
    if (X > 0.0)
        return (Delta * (long((X / Delta) + 0.5)));
    else
        return (Delta * (long((X / Delta) - 0.5)));
}
```

Файл methods.h

```
#include "methods.cpp"

extern double F(double);
double Round (double X,double Delta);
double HORDA(double Left, double Right, double Eps, int &N);
```